FITTRACK

Documentacion tecnica del proyecto

Aplicacion multiplataforma para seguimiento fitness, rutinas, progreso y administracion.

Guia para desarrollo, operacion y despliegue

Version del proyecto: 1.0.0
Expo SDK 57 | React Native 0.86 | Node.js / Express | MySQL

23 de octubre de 2026

Acerca de esta guia

Esta documentacion describe la implementacion actual del repositorio: aplicacion Expo para web y Android, backend REST, base de datos MySQL, integraciones externas y procedimientos de entrega. Cuando el README historico difiere del codigo, esta guia refleja el comportamiento observado en el codigo fuente al 23 de octubre de 2026.

Los valores privados de archivos .env no se incluyen. Se documentan unicamente los nombres de variables y ejemplos con valores ficticios.

Contenido

1. Resumen y arquitectura
2. Funciones y roles
3. Estructura del repositorio
4. Requisitos e inicio local
5. Variables de entorno y seguridad
6. Modelo de datos
7. API REST propia
8. Integraciones externas
9. Despliegue y publicacion
10. Operacion, pruebas y solucion de problemas

1. Resumen y arquitectura

FitTrack permite a una persona registrar su cuenta, consultar y completar rutinas asignadas, guardar datos de progreso y administrar su perfil. Los entrenadores administran sus usuarios y crean rutinas; los administradores tienen capacidad global de gestion. La misma interfaz se ejecuta como sitio web y aplicacion Android Expo.

Componentes

Componente	Responsabilidad	Tecnologia
Cliente	Pantallas, navegacion, formularios, validacion basica y llamadas HTTP.	Expo SDK 57, React Native, TypeScript
API FitTrack	Autenticacion, permisos, reglas de negocio, persistencia y correo de verificacion.	Node.js, Express 5, mysql2, JWT, bcryptjs, Nodemailer
Base de datos	Usuarios, progreso, ejercicios, asignaciones, rutinas y bitacoras.	MySQL; esquema en database/schema.sql
Hosting web	Compila y sirve el bundle web desde Amplify.	AWS Amplify; configuracion en amplify.yml
API de produccion	Backend desplegado en EC2 y administrado por PM2 en el entorno actual.	Windows PowerShell / PM2

Flujo general

Navegador o Expo Go / APK -> cliente FitTrack -> API REST FitTrack -> MySQL. La web y la app tambien consumen directamente API Ninjas, Agenda de Viajes y CineTeca. La integracion de viajes obtiene un token OAuth desde el cliente; por eso sus credenciales no deben considerarse secretas si se mantienen como variables EXPO_PUBLIC_.

Zona horaria: la validacion de rutinas del dia y la clasificacion del progreso usan America/Guatemala en el backend. Puede cambiarse con APP_TIME_ZONE.

2. Funciones y roles

Modulo	Funciones principales
Inicio	Resumen y accesos a las funciones disponibles.
Rutinas	Lista por fecha (hoy, futuras y pasadas), detalle, instrucciones y registro de ejercicios completados.
Progreso	Peso, peso objetivo, altura y objetivo; IMC, historial de peso y actividad semanal.
Ejercicios	Consulta de biblioteca externa, busqueda y ejercicios personalizados.
Perfil	Informacion personal, estadisticas y cierre de sesion.
Usuarios (admin)	Consulta de cuentas y cambio de rol.
Entrenamiento	Asignar/liberar usuarios, consultar historial y asignar rutinas. Admin puede hacerlo globalmente.
Viajes (admin)	Consultar viajes programados por estado y rango de fechas; registrar nuevos viajes.
Peliculas (admin)	Consultar ranking por valoracion o visualizaciones y registrar reviews.

Permisos de cuenta

Rol	Acceso
usuario	Sus rutinas, progreso, perfil y ejercicios propios.
entrenador	Usuarios asignados, historial de progreso y creacion de rutinas para ellos.
administrador	Consulta de todos los usuarios y rutinas; cambio de roles; gestion global de entrenamiento y pantallas de viajes/peliculas.

El registro publico crea cuentas con rol usuario. Para promover la primera cuenta a administrador se usa una actualizacion SQL controlada (ver seccion API/administracion); no se permite asignar ese rol desde el registro publico.

3. Estructura del repositorio

```
FitTrack/
|-- App.tsx, index.ts          Entrada de la aplicacion
|-- app.json                   Identidad, icono, web y Android
|-- package.json               Dependencias y comandos cliente
|-- amplify.yml                Build de Amplify
|-- eas.json                   Perfiles EAS para APK / Android
|-- database/schema.sql        Esquema MySQL
|-- server/index.js            API Express y reglas de negocio
|-- server/init-db.js          Migraciones/inicializacion incremental
|-- scripts/start-all.js       Arranque local conjunto
|-- src/navigation/            Navegacion principal y de ejercicios
|-- src/screens/               Pantallas por rol y modulo
|-- src/components/            Controles y componentes reutilizables
|-- src/services/              Clientes HTTP y modelos de API
|-- src/utils/                 Utilidades de UI y fitness
|-- src/types/                 Tipos TypeScript
|-- assets/images/             Logo y recursos visuales
```

Comandos del paquete raiz

Comando	Uso
npm start	Inicia los procesos configurados por scripts/start-all.js.
npm run start:expo	Inicia Expo para el cliente.
npm run web	Inicia la version web en desarrollo.
npm run build	Exporta el cliente web a dist/ para hosting.
npm run server	Inicia el backend definido en server/.
npm run db:init	Ejecuta la inicializacion/migraciones incluidas.

Navegacion adaptable

En web de escritorio la navegacion lateral puede retraerse con el boton de menu; la pantalla seleccionada retrae la barra. En ventanas compactas y en telefono la navegacion se presenta en la parte inferior.

4. Requisitos e inicio local

Requisitos

- Node.js 22.13 o posterior es la referencia recomendada para Expo SDK 57 y el build de Amplify.

- npm y PowerShell en Windows; Git para clonar y sincronizar cambios.

- Servidor MySQL accesible y credenciales de desarrollo.

- Expo Go para pruebas en telefono; EAS CLI para construir el APK.

Instalacion

```
# Desde la raiz del proyecto
npm ci

# Instalar dependencias del backend
npm --prefix server ci

# Crear la base y sus tablas
# Ejecutar database/schema.sql en la instancia MySQL

# Copiar .env.example a .env y completar la configuracion local
Copy-Item .env.example .env

# Arrancar API y cliente segun el script existente
npm start
```

Arranque por separado

```
# Terminal 1: API
npm --prefix server start

# Terminal 2: Expo
npx expo start

# Web de desarrollo
npm run web

# Exportacion web de produccion
npm run build
```

URL local para cada dispositivo

Entorno	URL tipica de API
Web / mismo equipo	http://127.0.0.1:3000/api
Emulador Android	http://10.0.2.2:3000/api
Telefono en la misma red	http://IP-LAN-DE-TU-PC:3000/api

En un telefono, localhost refiere al telefono, no a la PC. La PC y el dispositivo deben poder comunicarse por la red; el firewall debe permitir el puerto de desarrollo. Para produccion se debe usar HTTPS.

5. Variables de entorno y seguridad

Configura las variables en .env local para desarrollo y en el mecanismo de secretos del hosting para cada entorno. No compartas ni subas el archivo .env. El valor de DATABASE_URL puede estar presente por compatibilidad con la configuracion existente; el pool MySQL actual utiliza DB_HOST, DB_PORT, DB_NAME, DB_USER y DB_PASSWORD.

Cliente Expo (EXPO_PUBLIC_)

Variable	Proposito
EXPO_PUBLIC_DATABASE_API_URL	URL base de la API propia, normalmente termina en /api.
EXPO_PUBLIC_DATABASE_USER_ID	Identificador alternativo heredado para flujos sin sesion.
EXPO_PUBLIC_API_NINJAS_KEY	Clave usada por la biblioteca de ejercicios API Ninjas.
EXPO_PUBLIC_TRAVEL_OAUTH_URL	Origen del endpoint OAuth de Agenda de Viajes.
EXPO_PUBLIC_TRAVEL_API_URL	URL base del API de viajes.
EXPO_PUBLIC_TRAVEL_CLIENT_ID	Identificador del cliente OAuth de viajes.
EXPO_PUBLIC_TRAVEL_CLIENT_SECRET	Se usa para solicitar el token OAuth de viajes.
EXPO_PUBLIC_MOVIES_API_URL	URL base de la API CineTeca.

Backend privado

Variable	Proposito
API_PORT	Puerto HTTP de Express; valor por defecto 3000.
JWT_SECRET	Clave privada de firma JWT. Debe ser larga y aleatoria.
DB_HOST / DB_PORT	Host y puerto de MySQL; puerto por defecto 3306.
DB_NAME / DB_USER / DB_PASSWORD	Base de datos y credenciales MySQL.
APP_TIME_ZONE	Zona para las reglas de fechas; por defecto America/Guatemala.
SMTP_HOST / SMTP_PORT / SMTP_SECURE	Servidor SMTP y transporte TLS.
SMTP_USER / SMTP_PASSWORD	Cuenta y credencial SMTP del backend.
EMAIL_FROM	Remitente mostrado en los correos; usa SMTP_USER si no se define.

ADVERTENCIA: todo valor EXPO_PUBLIC_ se incorpora al bundle web y puede extraerse de una APK. El CLIENT_SECRET de viajes y la clave API de terceros no son secretos seguros en el cliente. Para produccion, mover OAuth y llamadas con secretos a endpoints proxy del backend, rotar credenciales expuestas y restringir las claves del proveedor cuando sea posible.

6. Modelo de datos

El esquema inicial esta en database/schema.sql. Las claves foraneas eliminan en cascada los datos dependientes cuando se elimina una cuenta, rutina o ejercicio relacionado.

Tabla	Contenido / relaciones
users	Cuenta, correo unico, hash de contrasena y rol.
registration_verifications	Verificacion de email pendiente; hash del codigo, vencimiento e intentos.
user_progress	Peso actual, peso objetivo, altura y objetivo; una fila por usuario.
weight_history	Historial diario de peso, unico por usuario y fecha.
custom_exercises	Ejercicios creados por una cuenta, instrucciones y seguridad.
trainer_user_assignments	Relacion entrenador-usuario; cada usuario puede tener un entrenador asignado.
routines	Rutina asignada a usuario por entrenador, fecha programada y dificultad.
routine_exercises	Relacion y orden de ejercicios dentro de una rutina.
routine_completions	Registro de finalizacion de rutina.
exercise_logs	Detalle de peso, duracion y notas de cada ejercicio realizado.

Relaciones principales

```
users 1---1 user_progress
users 1---N weight_history
trainer (users) 1---N trainer_user_assignments N---1 user (users)
trainer (users) 1---N routines N---1 user (users)
routines N---N custom_exercises via routine_exercises
routines 1---N routine_completions 1---N exercise_logs
```

Las claves foraneas vinculan rutinas con el entrenador y el usuario. Los ejercicios vinculados por rutina son ejercicios personalizados del entrenador que la crea.

7. API REST propia

La URL base se configura como EXPO_PUBLIC_DATABASE_API_URL y debe incluir el prefijo /api. Los endpoints protegidos aceptan Authorization: Bearer <JWT>. Las respuestas de error suelen incluir { message: string }.

Metodo y ruta	Auth / rol	Uso
GET /health	Publico	Estado del servicio.
POST /auth/register/request-code	Publico	Solicitar verificacion por email.
POST /auth/register/verify	Publico	Validar codigo y crear cuenta/sesion.
POST /auth/login	Publico	Iniciar sesion y recibir JWT.
GET /users	Administrador	Listar cuentas.
PATCH /users/:userId/role	Administrador	Cambiar rol.
GET /admin/routines	Administrador	Consultar todas las rutinas.
GET /trainer/users	Entrenador/Admin	Listar usuarios asignados o disponibles.
POST /trainer/users/:userId	Entrenador/Admin	Asignar usuario al entrenador.
DELETE /trainer/users/:userId	Entrenador/Admin	Liberar asignacion.
POST /trainer/users/:userId/routines	Entrenador/Admin	Crear rutina y asignarla.
GET /routines	Sesion	Rutinas del usuario autenticado.
POST /routines/:routineId/complete	Sesion	Completar rutina del dia actual.
GET /trainer/users/:userId/progress	Entrenador/Admin	Consultar actividad e historial del usuario.
GET /exercises/custom	Sesion	Listar ejercicios propios.
POST /exercises/custom	Sesion	Crear ejercicio personalizado.
DELETE /exercises/custom/:exerciseId	Sesion	Eliminar ejercicio propio.
GET /progress/:userId	Propietario	Leer progreso e historial.
PUT /progress/:userId	Propietario	Guardar progreso y peso.

Los endpoints de viajes y peliculas se consumen directamente desde el cliente en esta version; no son endpoints de la API FitTrack.

7.1 Registro, rutinas y progreso

Registro verificado

```
POST /api/auth/register/request-code
{ "name": "Nombre", "email": "persona@ejemplo.com", "password": "..." }

POST /api/auth/register/verify
{ "email": "persona@ejemplo.com", "code": "123456" }

# Registro confirmado devuelve { token, user: { id, name, email, role } }
```

La API exige una contrasena de al menos 8 caracteres, una mayuscula y un caracter especial. El codigo tiene 6 digitos, vence a los 10 minutos y admite hasta 5 intentos. La contrasena se almacena con bcrypt; el codigo se guarda como hash.

Crear rutina

```
POST /api/trainer/users/{userId}/routines
Authorization: Bearer <JWT>
Content-Type: application/json
{
  "title": "Fuerza - tren superior",
  "level": "Intermedio",
  "durationMinutes": 45,
  "scheduledDate": "2026-10-23",
  "exerciseIds": [12, 13]
}
```

Completar rutina

```
POST /api/routines/{routineId}/complete
Authorization: Bearer <JWT>
{
  "exercises": [
    { "exerciseId": "12", "weightKg": 20, "notes": "Buena tecnica" },
    { "exerciseId": "13", "durationMinutes": 25 }
  ]
}
```

La fecha debe ser hoy en America/Guatemala (configurable por APP_TIME_ZONE). Las rutinas pasadas o futuras no se pueden completar. Fuerza requiere peso y cardio requiere duracion; el registro se guarda en una transaccion.

Administrar y consultar progreso

```
# Cambiar rol (solo administrador)
PATCH /api/users/{userId}/role
{ "role": "entrenador" }

# Actualizar progreso del usuario autenticado
PUT /api/progress/{userId}
{ "weight": 75, "targetWeight": 70, "height": 1.72,
  "objective": "Perder grasa" }
```

Los roles permitidos son usuario, entrenador o administrador. Para establecer el primer administrador, confirma el correo y ejecuta con una cuenta MySQL autorizada:

```
UPDATE fittrack.users
SET role = 'administrador'
WHERE email = 'correo-confirmado@ejemplo.com';
```

La plantilla .env.example no incluye actualmente las variables SMTP; si se habilita el registro por correo, definelas en el entorno privado del backend (local o EC2), sin incorporarlas al bundle Expo.

8. Integraciones externas

Servicio	Operacion implementada	Configuracion
API Ninjas	Consulta ejercicios por nombre; la app normaliza y traduce algunos campos.	EXPO_PUBLIC_API_NINJAS_KEY
Agenda de Viajes	OAuth client_credentials; lista viajes por estado/fechas y crea viajes.	TRAVEL_OAUTH_URL, TRAVEL_API_URL, TRAVEL_CLIENT_ID, TRAVEL_CLIENT_SECRET
CineTeca	Ranking de peliculas por valoradas o vistas y creacion de review.	EXPO_PUBLIC_MOVIES_API_URL

Agenda de Viajes

```
POST {TRAVEL_OAUTH_URL}/oauth/token
Content-Type: application/x-www-form-urlencoded
grant_type=client_credentials
client_id=...
client_secret=...
scope=integraciones.programados.read integraciones.viajes.write

GET {TRAVEL_API_URL}/integraciones/v1/programados/viajes
    ?per_page=50&estado=planeado&fecha_desde=AAAA-MM-DD&fecha_hasta=AAAA-MM-DD
POST {TRAVEL_API_URL}/integraciones/v1/programados/viajes
```

El cuerpo de creacion contiene destino, pais, fecha_inicio, fecha_fin, descripcion, estado (planeado o en curso), presupuesto y moneda. Fechas se envian como YYYY-MM-DD.

CineTeca

```
GET {MOVIES_API_URL}/publico/peliculas/ranking?orden=valoradas&limit=20
GET {MOVIES_API_URL}/publico/peliculas/ranking?orden=vistas&limit=20
POST {MOVIES_API_URL}/publico/peliculas/{movieId}/reviews
{ "autor": "Nombre", "calificacion": 9, "comentario": "Comentario" }
```

Notas operativas

- Los dominios externos deben permitir las solicitudes del origen web si aplican politicas CORS.

- Errores 401/403 de viajes suelen indicar credenciales, scope o permisos; 404 suele indicar ruta/base URL incorrecta.

- La aplicacion conserva la fecha de expiracion del token de viajes en memoria y vuelve a solicitarlo cuando vence.

9. Despliegue y publicacion

Web mediante AWS Amplify

- El archivo amplify.yml usa Node 22, ejecuta npm ci y npm run build; publica dist/.

- Configura las variables EXPO_PUBLIC_ necesarias en el entorno/ramificacion que construye el sitio.

- Tras cambiar variables de build, inicia una nueva implementacion; editar variables no modifica el bundle ya publicado.

- Los cambios solo de backend no requieren rebuild web, salvo que cambie el cliente o una variable incrustada.

Backend en EC2 / PM2 (Windows PowerShell)

```
# En la carpeta del repositorio, tras verificar Git y Node
Set-Location C:\FitTrack
git pull origin master

# Instalar dependencias backend si package-lock.json cambio
npm --prefix server ci

# Reiniciar la API existente y conservar el entorno de EC2
pm2.cmd restart fittrack-api --update-env
pm2.cmd status
pm2.cmd logs fittrack-api --lines 100
pm2.cmd save
```

No sobrescribas el .env de EC2 al actualizar el codigo. Conserva alli las credenciales privadas DB, JWT y SMTP. Comprueba que PM2 apunta a server/index.js y valida GET /api/health. Si el ejecutable no se encuentra, usa su ruta completa o corrige PATH y abre una PowerShell nueva.

Android APK con EAS

```
# Desde la raiz del proyecto y con variables para el perfil elegido
npx eas-cli login
npx eas-cli build --platform android --profile preview
```

El perfil preview de eas.json configura distribution interna y android.buildType apk. Para publicar en tienda se utiliza el perfil production, que produce app-bundle; no es un APK de prueba. Las variables EXPO_PUBLIC_ deben estar en el entorno EAS asociado al perfil antes de comenzar el build.

Luego descarga el APK desde el enlace de EAS e instalalo en Android. La APK incorpora la URL de API y las variables publicas presentes al momento de compilar; un cambio de API del cliente requiere construir una nueva APK.

10. Operacion, verificacion y problemas comunes

Lista de verificacion

- API: GET /api/health responde correctamente desde la red publica y desde el cliente.

- Base de datos: reglas de entrada permiten conectar EC2 al puerto MySQL desde la IP/SG correcto; evita abrir MySQL al mundo.

- Autenticacion: registro envia codigo; verificacion crea usuario; login devuelve token y rol.

- Rutinas: el usuario ve solo las asignadas; una rutina de hoy permite registrar los datos requeridos por tipo de ejercicio.

- Administrador: puede consultar rutinas globales y gestionar usuarios; entrenador solo administra usuarios asignados.

- Web: probar resolucion movil/tablet/escritorio, CORS para integraciones y variables de Amplify.

- Android: probar la APK real instalada, conectividad HTTPS, login, progreso y permisos por rol.

Problemas frecuentes

Sintoma	Revision sugerida
No se reconoce node/npm/git/pm2	Verificar instalacion y PATH; abrir shell nueva. En Windows, probar pm2.cmd.
HTTP 404	Confirmar URL base, prefijo /api, ruta y que el backend nuevo fue desplegado.
HTTP 401/403	Iniciar sesion de nuevo; revisar JWT y rol/permisos de la cuenta.
No conecta a la API	Revisar health, HTTPS, DNS, security group, firewall y logs PM2.
No llega el codigo de correo	Revisar SMTP_HOST/PORT/SECURE/USER/PASSWORD y EMAIL_FROM en el backend; reiniciar PM2 con update-env.
Viajes no obtiene token	Comprobar URL OAuth, client ID, secret, scope y si la ruta /oauth/token responde.
Browser bloquea por CORS	Configurar el origen web en el servidor externo; usar HTTPS y no confundir URL del API con origen.
Rutina de hoy aparece como futura	Verificar APP_TIME_ZONE y que backend actualizado use America/Guatemala en lugar de CURRENT_DATE MySQL.

Comprobaciones de desarrollo

```
# Verificar syntax del servidor
node --check server/index.js

# Verificar tipos TypeScript (si el comando local esta disponible)
npx tsc --noEmit

# Crear build web local
npm run build
```

11. Seguridad, limites y mantenimiento

- Nunca publiques .env, contrasenas, tokens, claves OAuth ni exportaciones con credenciales. Si una clave se compartio por chat o se incluyo en un bundle, considerala expuesta y rotala.

- EXPO_PUBLIC_ no es un almacen de secretos. Mover operaciones con credenciales de terceros a la API FitTrack y guardar secretos unicamente en EC2/secret manager.

- En produccion: HTTPS, JWT_SECRET robusto, DB con privilegios minimos, MySQL privado por security group y copias de seguridad verificadas.

- Limitar CORS a origenes conocidos en APIs propias; hoy la API configura cors() abierto por defecto, conviene restringirlo antes de exponer nuevas operaciones sensibles.

- Agregar rate limiting a login, solicitud/verificacion de codigos y endpoints de escritura; revisar retencion/limpieza de codigos expirados.

- La configuracion SMTP pertenece al backend, no a Expo. Un valor EMAIL_FROM distinto del remitente autorizado puede provocar rechazo del proveedor.

- Documentar cambios de esquema con una migracion repetible y backup antes de alterar produccion.

Limites conocidos en la implementacion observada

La documentacion describe las rutas presentes en server/index.js y el cliente. Las APIs externas dependen de disponibilidad, contratos, CORS, credenciales y limites definidos por sus proveedores, que pueden cambiar sin modificar este repositorio. No se incluyen tokens ni datos de usuarios.

Fuentes internas consultadas

README.md, package.json, app.json, amplify.yml, eas.json, database/schema.sql, server/index.js, server/init-db.js y los modulos de src/services, src/navigation y src/screens. Esta guia no sustituye las politicas ni contratos vigentes de AWS, Expo o de las APIs externas.

Fin de la documentacion